Module 5:

Advanced Server-Side Development

5.1 JavaScript Pseudo-Classes

Although JavaScript has no formal class mechanism, it does support. While most object-oriented languages that support objects also support classes formally, JavaScript does not.Instead, you define pseudo-classes through a variety of interesting and nonintuitive syntax constructs. Many common features of object-oriented programming, such as inheritance and even simple methods, must be arrived at through these nonintuitive means. From a practical perspective, almost all modern frameworks (such as jQuery and the Google Maps API) use prototypes to simulate classes, so understanding the mechanism is essential to apply those APIs in your applications.

Using Object Literals Without Classes

An object can be instantiated using object literals. Object literals are a list of key-value pairs with colons between the key and value with commas separating key-value pairs. Object literals are also known as Plain Objects in jQuery.

```
<html>
  <body>
    <script>
        emp= {
                id:102,
                name:"Shyam Kumar",
                salary:40000
            }
        document.write("<strong>Employee ID:</strong>"+emp.id+" "+"<strong>Employee
                Name:</strong>"+emp.name+" "+"<strong>Employee  Salary:</strong>“ +emp.salary);
    </script>

  </body>
```

Example 2

```
<html>
  <body>
    <script>
      car = {
              brand: "Tesla",
              model: "Model 3",
                year: 2022
              };
```

```
document.write("car brand name:"+car.brand+"<br>");
document.write("car Model:"+car.model+"<br>");
document.write("car manufactured year:"+car.year+"<br>");
      </script>
   </body>
</html>
```

car brand name:Tesla
car Model:Model 3
car manufactured year:2022

Consider a die (single dice) object with dice number stored in array structure:

```
<html>
   <body>
      <script>
       var oneDie =      { color : "red",
                    faces : [1,2,3,4,5,6]
                  };
       document.write("die color:"+oneDie.color+"<br>");
       document.write("Die Faces:"+oneDie.faces[1]+"<br>");
      </script>

   </body>
</html>
```

Emulate Classes with functions by including variable

Although a formal class mechanism is not available to us in JavaScript, it is possible to get close by using functions to encapsulate variables and methods together, as shown

```
<html>
   <body>
      <script>

                        function Die(col){
                        this.color=col;
                        this.faces = [1,2,3,4,5,6];
                        }


                var col=prompt("enter Die color","");
                var oneDie = new Die(col);
       document.write("die color:"+oneDie.color+"<br>");
       document.write("Die Faces:"+oneDie.faces[1]+"<br>");
      </script>
   </body>
</html>
```

Very simple **Die pseudo-class definition as a function**

Adding Methods to the Object:

Die pseudo-class with an internally defined method

```
<html>
  <body>
    <script>

                function Die(col) {
                this.color=col;
                this.faces=[1,2,3,4,5,6];

 // define method randomRoll as an anonymous function or inline method
                    this.randomRoll = function()
                                {
                       var randNum = Math.floor((Math.random() * this.faces.length)+ 1);
                        return (this.faces[randNum -1]);
                                };
                }

            var col=prompt("enter Die color","");
            var oneDie = new Die(col);
              document.write("die color:"+oneDie.color+"<br>");
              document.write("Die Faces:"+oneDie.randomRoll()+" was rolled"+"<br>");
             </script>
  </body>
</html>
```

Drawback of Inline method

- Although this mechanism for methods is effective, it is not a memory-efficient approach because each inline method is redefined for each new object.
- Just imagine if you had 100 Die objects created; you would be redefining every method 100 times, which could have a noticeable effect on client execution speeds and browser responsiveness. This needless waste of memory

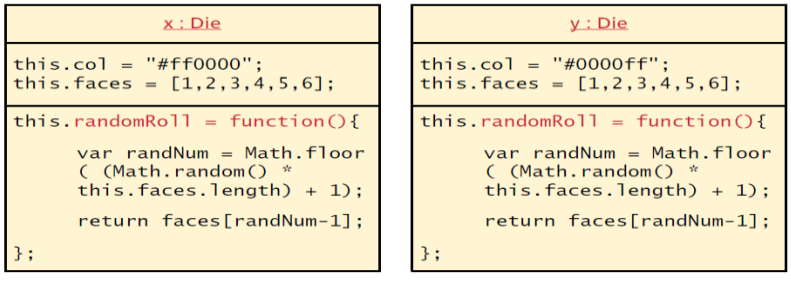


FIGURE 15.1 Illustrating duplicated method definition

Using Prototypes

To prevent this needless waste of memory, a better approach is to define the method just once using a *prototype* of the class.

So you can use a *prototype* of the class.

- **Prototypes are used to make JavaScript behave more like an object-oriented language.**
- The prototype properties and methods are defined *once* for all instances of an *object*.
- Every object has a prototype

```
<html>
  <body>
    <script>


            // Start Die Class
      function Die(col) {
            this.color=col;
            this.faces=[1,2,3,4,5,6];
              }
   Die.prototype.randomRoll = function() {
   var randNum = Math.floor((Math.random() * this.faces.length) + 1);
   return (this.faces[randNum-1]);
            };
  // End Die Class
var col=prompt("enter Die one color","");
 var oneDie = new Die(col);
 document.write("die one color:"+oneDie.color+"<br>");
 document.write("Die one Faces:"+oneDie.randomRoll()+"
     was rolled"+"<br>");


  var col1=prompt("enter Die two color","");
  var twoDie = new Die(col1);
  document.write("die two color:"+twoDie.color+"<br>");
```

```
    document.write("Die two Faces:“ +twoDie.randomRoll()+"
       was rolled"+"<br>");
     </script>
   </body>
</html>
```

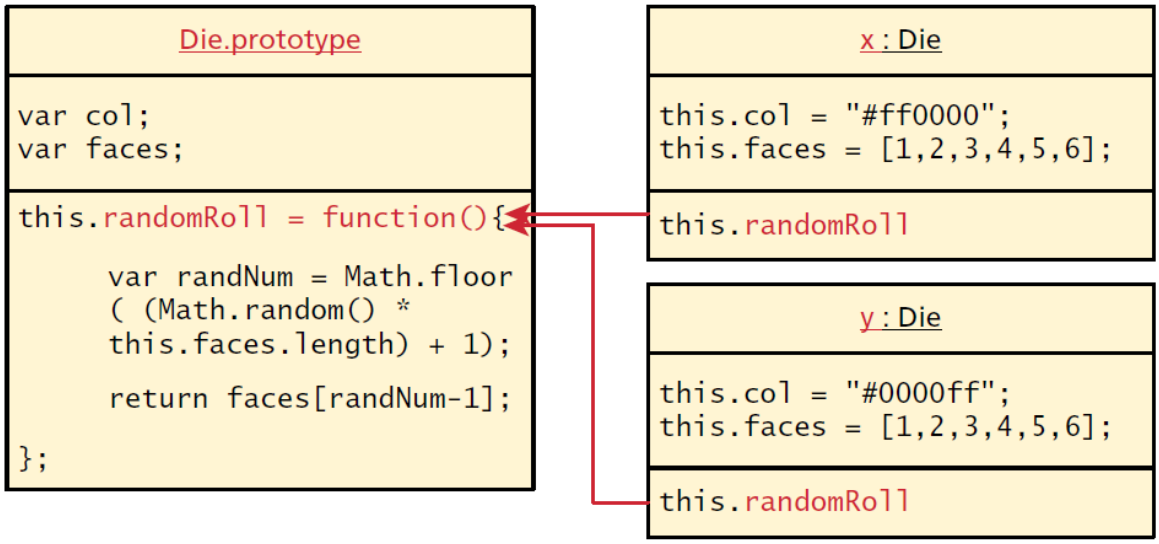


FIGURE 15.2 Illustration of JavaScript prototypes as pseudo-classes

Adding method to Prototype or Extending Built-in Objects

Adding a method to a prototype allows all instances of an object to share the same functionality.

The method is defined once on the prototype.

```
<html>
   <body>
      <script>
let a1 = [1, 2, 3, 4, 5];
let a2 = [5, 6, 7, 8, 9];
Array.prototype.sum = function () {
   let sum = 0
   for (let i = 0; i < this.length; i++) {
      sum += this[i]
   }
   return sum
};
document.write("the sum of a1 array is "+a1.sum()+"<br>");
document.write("the sum of a2 array is "+a2.sum()+"<br>");
</script>
   </body>
```

```
</html>
```

```
the sum of a1 array is 15
the sum of a2 array is 35
```

5.2 jQuery Foundations

Introduction jQuery **framework:**

- A jQuery **library** or **framework** is software that you can utilize in your own software, which provides some common implementations of standard ideas.
- jQuery is a fast, lightweight, and feature-rich JavaScript library designed with the core philosophy of "Write Less, Do More". Released in 2006 by John Resig,
- it provides an easy-to-use API that simplifies complex tasks like HTML document traversal, event handling, animation, and Ajax interactions across a wide variety of web

Key Advantages of jQuery

- **Simplicity and Conciseness:** jQuery takes common tasks that require many lines of vanilla JavaScript code and wraps them into methods that can be called with a single line. For example, instead of using document.getElementById('college').innerHTML = 'text', you can simply use $('#college').html('text').
- **Cross-Browser Compatibility:** One of its biggest strengths is handling browser inconsistencies. Code written in jQuery is designed to run exactly the same way in all major modern browsers, saving developers from writing specific "hacks" for different platforms
- **Efficient DOM Manipulation:** It provides powerful CSS-like selectors (e.g., by ID, class, or attribute) to easily find, select, and modify HTML elements and their styles.
- **Built-in Animations and Effects:** jQuery includes pre-built methods to create complex animations like sliding, fading, and toggling visibility with minimal code.
- **Simplified Ajax:** It makes interacting with servers and updating content without refreshing the page much easier compared to standard JavaScript.
- **Extensive Plugin Ecosystem:** There is a vast library of thousands of plugins created by a large community, allowing you to easily add advanced features like image sliders, form validation, and date pickers to your site.
- Many of the biggest companies on the Web use jQuery, such as:
    - Google
    - Microsoft
    - IBM
    - Netflix

Including jQuery in Your Page OR Adding jQuery to Your Web Pages:

There are several ways to start using jQuery on your web site. You can:

1. Download the jQuery library from jQuery.com
2. Include jQuery from a CDN, like Google

Downloading jQuery

There are two versions of jQuery available for downloading:

- Production version - this is for your live website because it has been minified and compressed
- Development version - this is for testing and development (uncompressed and readable code)
- Both versions can be downloaded from jQuery.com.

1. ONCE DOWNLOAD THE FILE IN YOUR SYSTEM THEN YOU CALL jQuery library following method.

Place the *jquery.min.js* file in a directory within your project, such as js/.

The jQuery library is a single JavaScript file, and you reference it with the HTML <script> tag (notice that the <script> tag should be inside the <head> section):

Syntax:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <title>jQuery Page</title>

     <!-- Local jQuery -->
     <script src="js/jquery.min.js"></script>
</head>

<body>
   <!-- Page Content -->
</body>
</html>
```

2. jQuery CDN

If you don't want to download and host jQuery yourself, you can include it from a **CDN (Content Delivery Network).**

Google is an example of someone who host jQuery:

Syntax:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <title>jQuery Page</title>
   <!-- jQuery CDN Link -->
  <script  src=  "https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js">
</script>
</head>

<body>
  <!-- Page Content -->
</body>
</html>
```

Basic Syntax for jQuery Function

In jQuery, the syntax for selecting and manipulating HTML elements is written as:

```
$(selector).action()
```

Where -

- **$ -** This symbol is used to access jQuery. It's a shorthand for the jQuery function.
- **(selector) -** It specifies which HTML elements you want to target. The selector can be any valid CSS selector, such as a class, ID, or tag name.
- **action() -** It represents the action or method that you want to perform on the selected elements. Common actions include manipulating CSS properties, handling events, or performing animations.

Example:

Develop a jQuery program that enables users to change the color of the content in the h1 element.

```
<!DOCTYPE html>
<html>
  <head>
    <title> jQuery </title>
    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
  </head>
<body>
  <h1>jQuery Basics</h1>
</body>
<script>
  $("h1").css("color", "red");
</script>
</html>
```

Output:

Document Ready Event:

- This is to prevent any jQuery code from running before the document is finished loading (is ready).
- It is good practice to wait for the document to be fully loaded and ready before working with it. This also allows you to have your JavaScript code before the body of your document, in the head section.

- **$(document).ready(function(){**

  // jQuery methods go here...

  });

Example:

Develop a web page using jQuery that contains a heading, two paragraphs, and a button. When the user clicks the button, the paragraph with the id test should be hidden from the page.

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#test").hide();
  });
});
</script>
</head>
<body>
<h2>This is a heading</h2>
<p>This is a paragraph.</p>
<p id="test">This is another paragraph.</p>
<button>Click me</button>
</body>
</html>
```

jQuery Selectors:

he relationship between DOM objects and selectors is so important in JavaScript programming that the **pseudo-class** bearing the name of the framework,

jQuery()

Is reserved for selecting elements from the DOM.
Because it is used so frequently, it has a shortcut notation and can be written as

$()

Basic Selectors:

The four basic selectors are:

- $("*") **Universal selector** matches all elements (and is slow).
- $("tag") **Element selector** matches all elements with the given element name.
- $(".class") **Class selector** matches all elements with the given CSS class.
- $("#id") **Id selector** matches all elements with a given HTML id attribute.

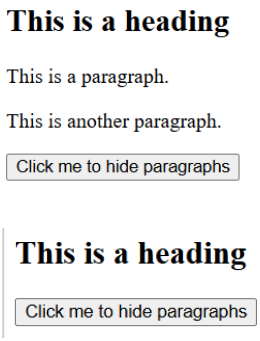
1. The element Selector

You can select all <p> elements on a page like this:

$("p")

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
```

```
    $("p").hide();
  });
});
</script>
</head>
<body>
<h2>This is a heading</h2>
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
<button>Click me to hide paragraphs</button>
</body>
</html>
```



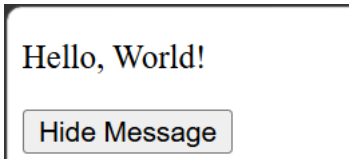
2. The #id Selector

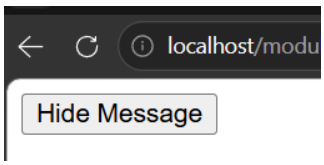
The jQuery *#id* selector uses the id attribute of an HTML tag to find the specific element.
An id should be unique within a page, so you should use the #id selector when you want to find a single, unique element.
To find an element with a specific id, write a hash character, followed by the id of the HTML element:

Example:
Create a basic HTML page with a <button> and a <p> element. When the user clicks the button, the content of the paragraph will be hidden.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My First jQuery Example</title>
  <!-- Include jQuery from CDN Link -->
  <script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js">
  </script>
</head>
<body>
  <p id="message">Hello, World!</p>
```

```
    <button id="hideButton">
      Hide Message
    </button>
    <!-- jQuery Script -->
    <script>
      $(document).ready(function()
          {
            $('#hideButton').click(function()
              {
                  $('#message').hide();
              });
          });
</script>
</body>
</html>
```





3. jQuery .Class Selector

In jQuery, a class selector is used to target HTML elements by their class attribute using the dot (.) symbol followed by the class name.

Syntax:

$(".class")

Example:

Develop a web page that uses jQuery class selectors and the css() method to modify the background color of a paragraph dynamically on a button click event.

```
<!DOCTYPE html>
<html>
<head>
  <!--jQuery library included -->
  <script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
  </script>

  <script>
    $(document).ready(function () {
        $(".colorBtn").click(function () {
        $(".p1").css("background-color", "yellow");
```

```
        });
      });
  </script>
</head>
<body>
  <h2>Class selector</h2>
  <p class="p1">
    In this peragraph we are changing background.
  </p>
  <!-- Button to change background
    color of the first paragraph -->
  <button class="colorBtn">
    Change Background Color
  </button>
</body>
</html>
```

Class selector

In this peragraph we are changing background.

Change Background Color

Class selector

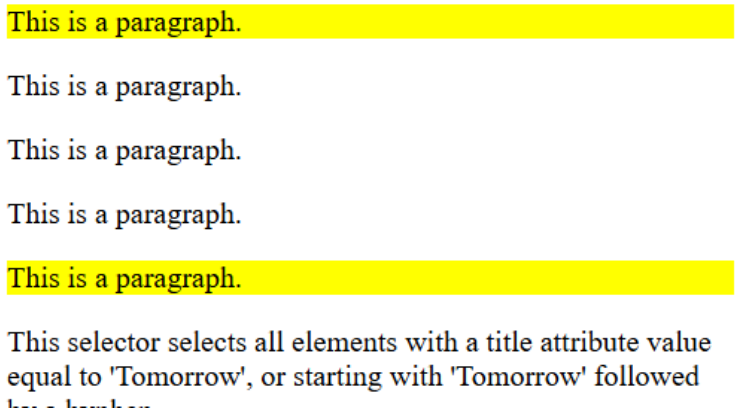
In this peragraph we are changing background.

Change Background Color

4. jQuery attribute Selector

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("p[title|='Tomorrow']").css("background-color", "yellow");});
</script>
</head>
<body>
<p title="Tomorrow">This is a paragraph.</p>
<p title="tomorrow">This is a paragraph.</p>
<p title="Tom">This is a paragraph.</p>
<p title="See You Tomorrow">This is a paragraph.</p>
<p title="Tomorrow-the day after today">This is a paragraph.</p>
<p>This selector selects all elements with a title attribute value equal to 'Tomorrow', or starting with 'Tomorrow' followed by a hyphen.</p>
```

```
</body>
</html>
```



5. Pseudo-Element Selector

Finds the first span in each matched div to underline and add a hover state.

Example:

Create a web page with multiple elements, each containing several elements. Using jQuery, select the first child element of every and underline its text. When the user moves the mouse over the selected element, change its color to green and make the text bold. Restore the original style when the mouse pointer leaves the element.

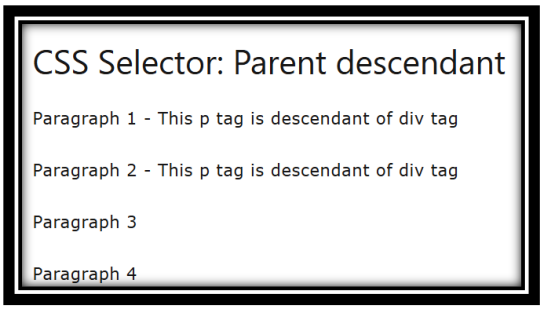
```
<!doctype html>
<htmllang="en">
<head>
 <metacharset="utf-8">
 <title>first-child demo</title>
 <style>
 span {
  color: #008;
 }
 span.sogreen {
  color: green;
  font-weight: bolder;
 }
 </style>
 <scriptsrc="https://code.jquery.com/jquery-4.0.0.js"></script>
</head>
<body>
<div>
 <span>John,</span>
 <span>Karl,</span>
 <span>Brandon</span>
</div>
<div>
 <span>Glen,</span>
 <span>Tane,</span>
```

```
  <span>Ralph</span>
</div>
<script>
$( "div span:first-child" )
  .css( "text-decoration", "underline" )
  .hover(function() {
    $( this ).addClass( "sogreen" );
  }, function() {
    $( this ).removeClass( "sogreen" );
  });
</script>
</body>
</html>
```

6. Contextual Selector

Write a jQuery program using the descendant selector (div p) to select all <p> elements inside elements and append a message to them."

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"></script>
<script>
    $(document).ready(function () {
          $('div p').append(" - This p tag is descendant of div tag");
                });
</script>
</head>
<body>          <
h1>CSS Selector: Parent descendant</h1>
<div>
     <p>Paragraph 1</p>
     <div>
        <p>Paragraph 2 </p>
    </div>
</div>
<p>Paragraph 3</p>
<span>
     <p>Paragraph 4</p>
</span>
</body>
</html>
```



7. Content Filter

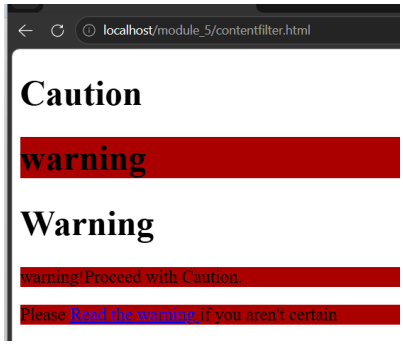
The **content filter** is the only jQuery selector that allows you to append filters to all of the selectors you've used thus far and match a particular pattern. You can select elements that have a particular child using :has(), have no children using :empty, or match a particular piece of text with :contains(). Consider the following example:

Example:

Create a web page containing different HTML elements such as headings, paragraphs, and hyperlinks. Use jQuery to search for elements that contain the text "warning" and change their background color dynamically when the page loads.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <title>My First jQuery Example</title>
   <!-- Include jQuery from CDN Link -->
   <script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js">
   </script>
</head>
<body>
   <h1>Caution</h1>
   <h1>warning</h1>
   <h1>Warning</h1>
   <p>warning!Proceed with Caution.</p>
<p>Please
<a href='#'>Read the warning </a>
if you aren't certain
</p>
<!-- jQuery Script -->
   <script>
      $(document).ready(function() {
         $("body *:contains('warning')").css("background-color", "#aa0000");

      });
   </script>
</body>
</html>
```



8. Form Selectors

Since form HTML elements are well known and frequently used to collect and transmit data, there are jQuery selectors written especially for them. These selectors, listed in

Selector	Matches Elements	Standard CSS Equivalent
`:input`	All `<input>`, `<textarea>`, `<select>`, and `<button>` elements	`input, textarea, select, button`
`:text`	`<input type="text">` elements	`input[type="text"]`
`:password`	`<input type="password">` elements	`input[type="password"]`
`:radio`	All radio button inputs	`input[type="radio"]`
`:checkbox`	All checkbox inputs	`input[type="checkbox"]`
`:submit`	Submit buttons and inputs with `type="submit"`	`input[type="submit"], button[type="submit"]`
`:button`	Any button element or input with `type="button"`	`button, input[type="button"]`

Example:
Create a feedback form containing various input controls such as a text box, password field, drop-down list, radio buttons, and checkboxes. Use jQuery selectors (:text, [type='password'], :selected, :checked, and :input) to retrieve the values entered or selected by the user and display them in a formatted output area when a button is clicked.

Form selector in jQuery

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
```

```
    <title>jQuery Form Selector Example</title>
    <!-- Loading jQuery Library via CDN -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
    <style>
        body { font-family: Arial, sans-serif; margin: 30px; line-height: 1.6; }
        .form-group { margin-bottom: 15px; }
        label { display: inline-block; width: 120px; font-weight: bold; }
        #output { margin-top: 20px; padding: 15px; background: #f4f4f4; border-left: 5px solid
#007bff; display: none; }
    </style>
</head>
<body>

    <h2>Feedback Form</h2>

    <form id="myForm">
        <!-- Text Input -->
        <div class="form-group">
            <label for="username">Username:</label>
            <input type="text" id="username" name="username" value="JohnDoe">
        </div>

        <!-- Password Input -->
        <div class="form-group">
            <label for="pass">Password:</label>
            <input type="password" id="pass" name="pass" value="secret123">
        </div>

        <!-- Dropdown Menu / Select Box -->
        <div class="form-group">
            <label for="country">Country:</label>
            <select id="country" name="country">
                <option value="us">United States</option>
                <option value="ca" selected>Canada</option>
                <option value="uk">United Kingdom</option>
            </select>
        </div>

        <!-- Radio Buttons -->
        <div class="form-group">
            <label>Subscription:</label>
            <input type="radio" name="subType" value="Free"> Free
            <input type="radio" name="subType" value="Premium" checked> Premium
        </div>

        <!-- Checkbox Input -->
        <div class="form-group">
```

```
        <label>Preferences:</label>
        <input type="checkbox" name="newsletter" value="Yes" checked> Subscribe to
Newsletter
    </div>

    <!-- Buttons -->
    <button type="button" id="processBtn">Read Form Data</button>
  </form>

  <!-- Output Box -->
  <div id="output">
    <h3>Selected Values:</h3>
    <div id="results"> </div>
  </div>

  <script>
    $(document).ready(function() {
      // Trigger selector operations on button click
      $("#processBtn").click(function() {

        // 1. Select text input by type pseudo-selector
        var txtVal = $("input:text").val();

        // 2. Select password input using attribute selector
        var passVal = $("input[type='password']").val();

        // 3. Select the actively highlighted element from a dropdown menu
        var selectVal = $("form select option:selected").val();

        // 4. Select the chosen radio button among a shared name group
        var radioVal = $("input[name='subType']:checked").val();

        // 5. Select checked boxes and determine if they are verified
        var checkVal = $("input[name='newsletter']").is(":checked") ? "Subscribed" : "Not
Subscribed";

        // 6. Count total form controls utilizing the general :input selector
        var totalInputs = $("#myForm :input").length;

        // Build a summary string
        var htmlSummary = "<ul>" +
          "<li><b>Username (input:text):</b> " + txtVal + "</li>" +
          "<li><b>Password (input[type='password']):</b> " + passVal + "</li>" +
          "<li><b>Country (option:selected):</b> " + selectVal + "</li>" +
          "<li><b>Subscription (:checked radio):</b> " + radioVal + "</li>" +
          "<li><b>Newsletter (:checked checkbox):</b> " + checkVal + "</li>" +
          "<li><b>Total Form Controls (:input count):</b> " + totalInputs + "</li>" +
```

```
            "</ul>";

            // Render summary output dynamically to the view layer
            $("#results").html(htmlSummary);
            $("#output").fadeIn();
        });
    });
  </script>

</body>
</html>
```

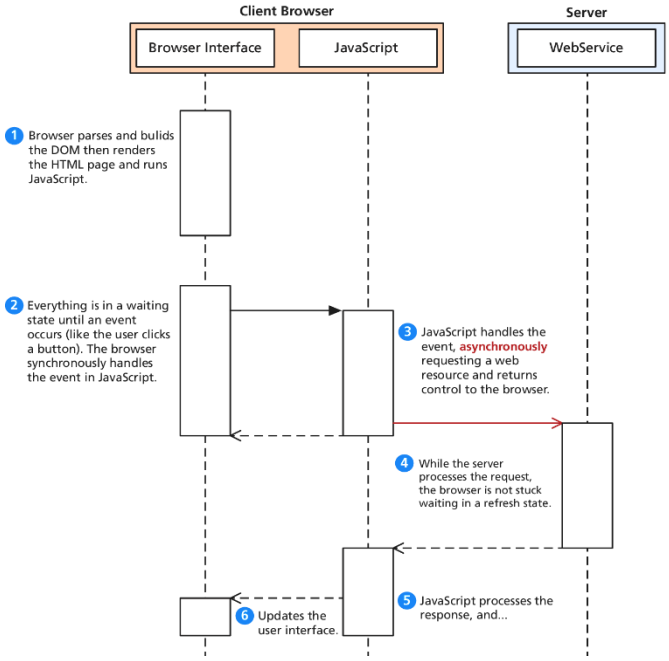
Shortcut Methods

- The **html()** method is used to get the HTML contents of an element. If passed with a parameter, it updates the HTML of that element.
- The **val()** method returns the value of the element.
- The shortcut methods **addClass**(className) / **removeClass**(className) add or remove a CSS class to the element being worked on. The className used for these functions can contain a space-separated list of classnames to be added or removed.
- The **hasClass**(classname) method returns true if the element has the className currently assigned. False, otherwise.

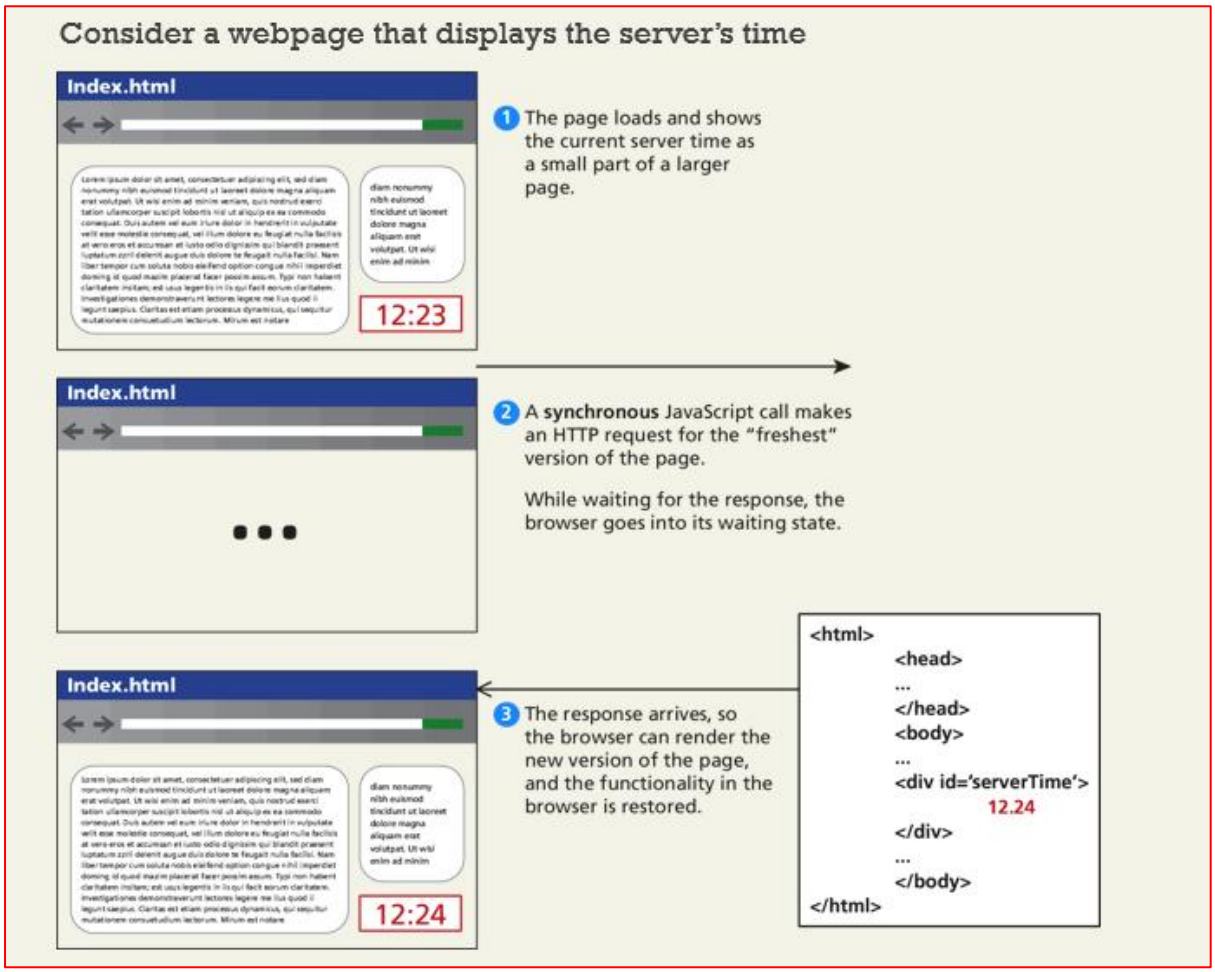
5.3 AJAX (Asynchronous JavaScript with XML)

- Asynchronous JavaScript with XML (AJAX) is a term used to describe a paradigm that allows a web browser to send messages back to the server without interrupting the flow of what's being shown in the browser.
- This makes use of a browser's multi-threaded design and lets one thread handle the browser and interactions while other threads wait for responses to asynchronous requests.

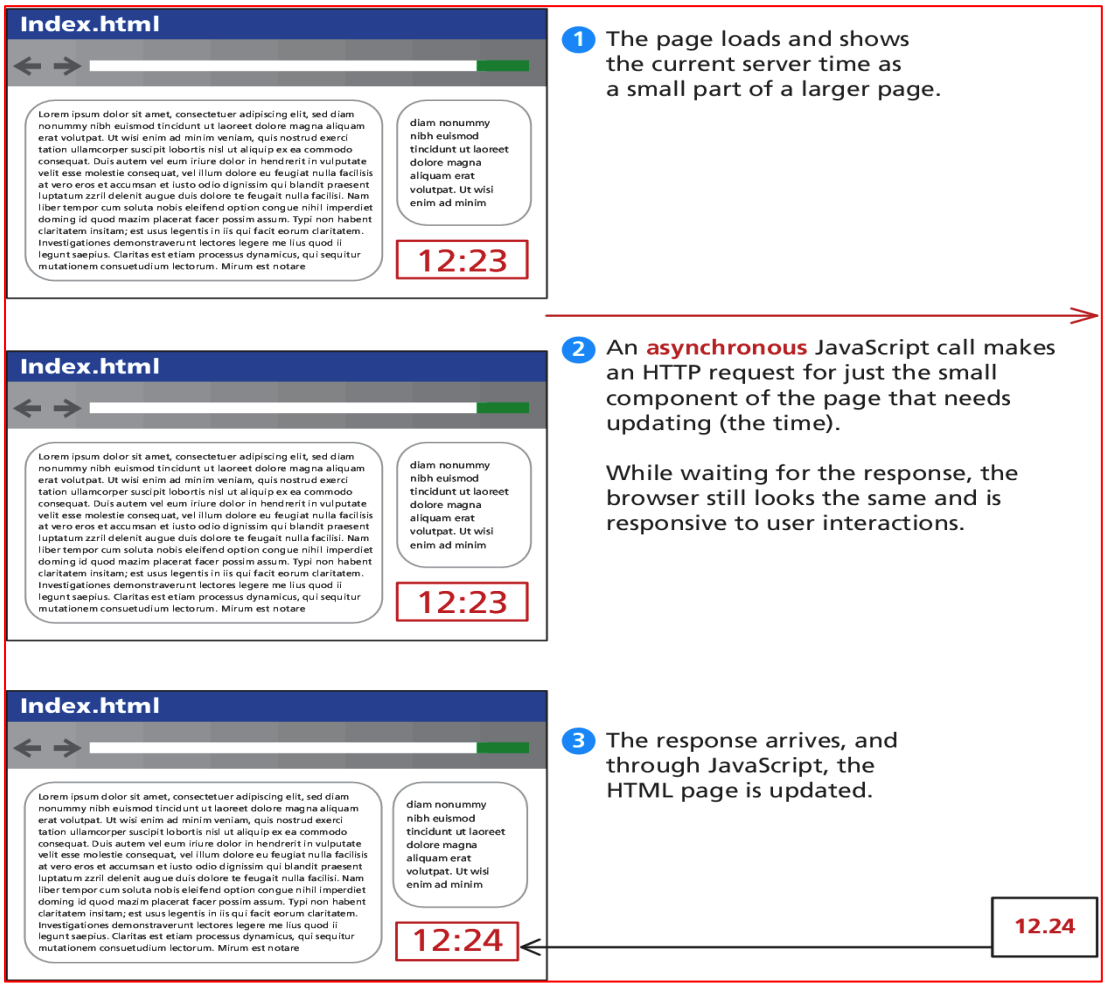
Figure 15.8 annotates a UML sequence diagram where the white activity bars illustrate where computation is taking place. Between the request being sent and the response being received, the system can continue to process other requests from the client, so it does not appear to be waiting in a loading state.



EXAMPLE WITHOUT USING AJAX TECHNIQUE BROWSER WILL WORK LIKE THIS



EXAMPLE WITH USING AJAX TECHNIQUE BROWSER WILL WORK LIKE THIS



How to Making Asynchronous requests using AJAX WITH jQuery

jQuery provides a family of methods to make asynchronous requests. We will start with the simplest GET requests, and work our way up to the more complex usage of AJAX where all variety of control can be exerted.

Consider for instance the very simple server time page described above. If the URL currentTime.php returns a single string and you want to load that value asynchronously into the element, you could write:

```
$("#timeDiv").load("currentTime.php");
```

Method 1: AJAX GET Requests

Scenario statement:

To illustrate the more powerful features of jQuery and AJAX, consider the more complicated scenario of a web poll where the user must choose one of the four options as illustrated

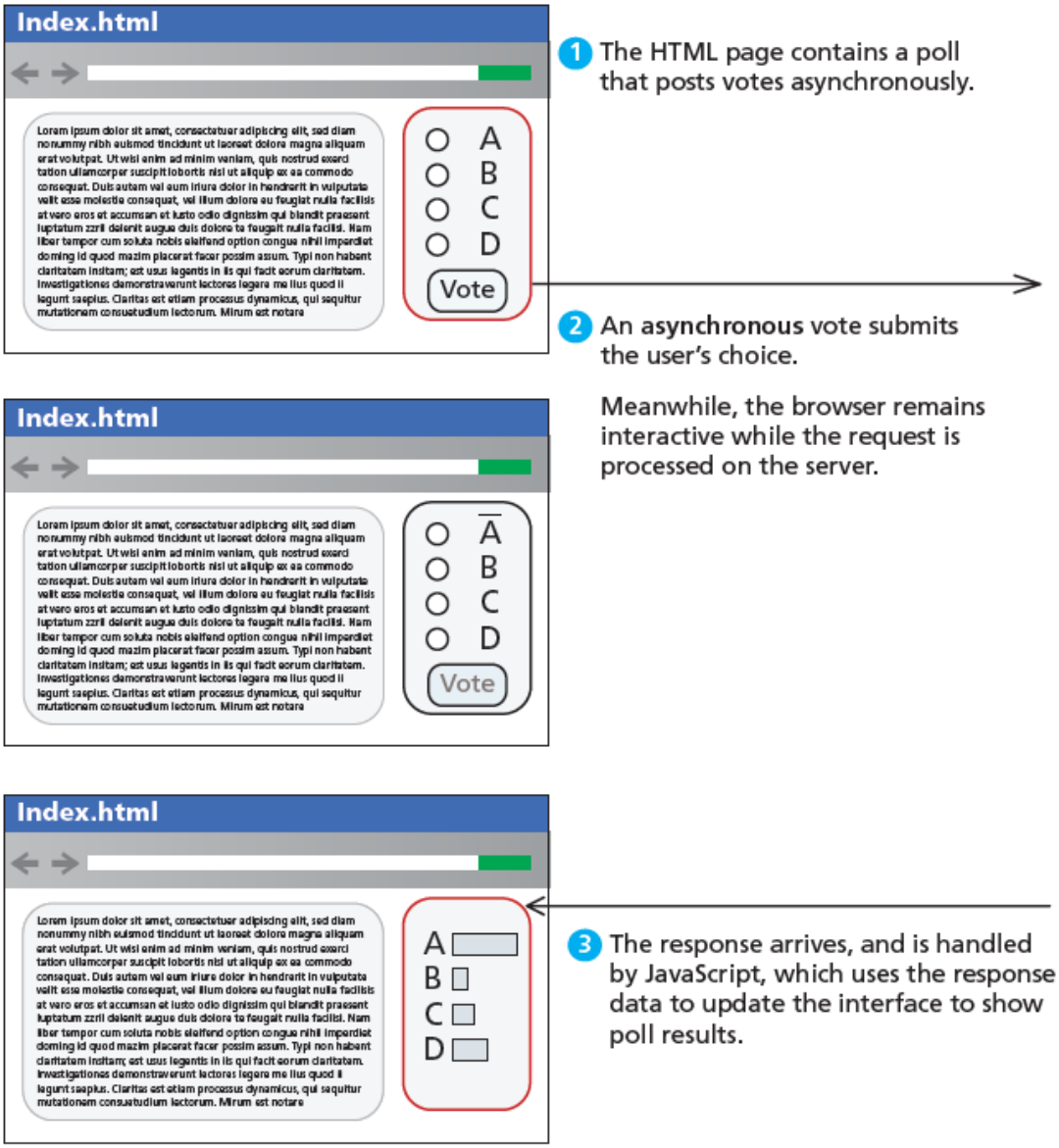


FIGURE 15.11 Illustration of a simple asynchronous web poll

Working of the Program

- User selects **A, B, C, or D**.
- Clicking **Vote** calls submitVote().
- AJAX (XMLHttpRequest) sends the selected option to poll.php.
- The page does **not refresh**.
- PHP updates the vote count and returns the latest results.
- JavaScript displays the results inside the result div dynamically.

Step 1:

Making a request to vote for option C in a poll could easily be encoded as a URL request GET /vote.php?option=C.

```
$.get("/vote.php?option=C");
```

Step 2:
The formal definition of the get() method lists one required parameter url and three optional ones: data, a callback to a success() method, and a dataType.

jQuery.get (url [, data] [, success(data, textStatus, jqXHR)] [, dataType])

jQuery.get (**url [, data] [, success(data, textStatus, jqXHR)] [, dataType]**)

- **url** is a string that holds the location to send the request.
- **data** is an optional parameter that is a query string or a *Plain Object.*
- **success(data,textStatus,jqXHR)** is an optional *callback* function that executes when the response is received.
    - **data** holding the body of the response as a string.
    - **textStatus** holding the status of the request (i.e., "success").
    - **jqXHR** holding a jqXHR object, described shortly.
- **dataType** is an optional parameter to hold the type of data expected from the server.

Step 3:
The callback function is passed as the second parameter to the get() method and uses the textStatus parameter to distinguish between a successful post and an error.

```
$.get("/vote.php?option=C", function(data,textStatus,jsXHR) {
if (textStatus=="success") {
console.log("success! response is:" + data);
}
else {
console.log("There was an error code"+jsXHR.status);
}
console.log("all done");
});
```

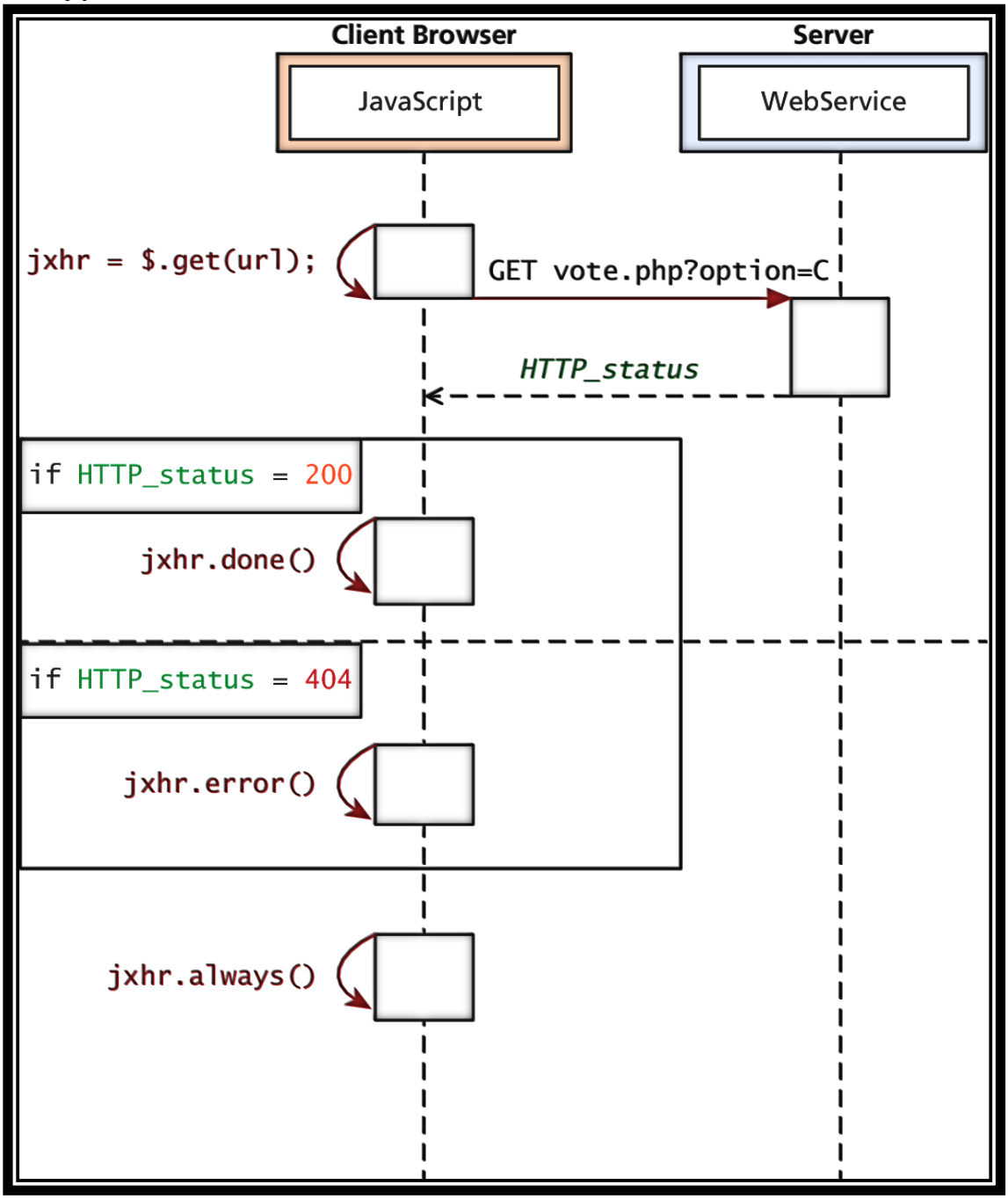
- Unfortunately, if the page requested (vote.php, in this case) does not exist on the server, then the callback function does not execute at all, so the code announcing an error will never be reached.
- To address this we can make use of the **jqXHR object** to build a more complete solution.

Step 4:
jqXHR - XMLHttpRequest compatibility

- **abort()** stops execution and prevents any callback or handlers from receiving the trigger to execute.
- **getResponseHeader()** takes a parameter and gets the current value of that header.
- **readyState** is an integer from 1 to 4 representing the state of the request. The values include 1: sending, 3: response being processed, and 4: completed.
- **responseXML** and/or **responseText** the main response to the request.

- **setRequestHeader(name, value)** when used before actually instantiating the request allows headers to be changed for the request.
- **status** is the HTTP request status codes (200 = ok)
- **statusText** is the associated description of the status code.

jqXHR



jqXHR objects have methods

- done()
- fail()
- always()
- which allow us to structure our code in a more modular way than the inline callback

```
var jqxhr = $.get("/vote.php?option=C");
jqxhr.done(function(data) { console.log(data);});
jqxhr.fail(function(jqXHR) { console.log("Error: "+jqXHR.status); })
jqxhr.always(function() { console.log("all done"); });
```

POST requests using jQuery

- POST requests are often preferred to GET requests because one can post an unlimited amount of data, and because they do not generate viewable URLs for each action.
- GET requests are typically not used when we have forms because of the messy URLs and that limitation on how much data we can transmit.
- Definition describes GET as a **safe method** meaning that they should not change anything, and should only read data. POSTs on the other hand are not safe, and should be used whenever we are changing the state of our system (like casting a vote).

Step 1:

Making a request to vote for option C in a poll could easily be encoded as a URL

request POST /vote.php?option=C.

```
$.POST("/vote.php?option=C");
```

Step 2:

The formal definition of the get() method lists one required parameter url and three optional ones: data, a callback to a success() method, and a dataType.

jQuery.post (url [, data] [, success(data, textStatus, jqXHR)] [, dataType])

Step 4: [receive the formdata to jQuery variable]

Since jQuery can be used to submit a form, you may be interested in the shortcut method serialize(), which can be called on any form object to return its current key-value pairing as an & separated string, suitable for use with post().

```
var postData = $("#voteForm").serialize();
```

Step 5: [form data encoded into query string and send to server file postData.php]

With the form's data now encoded into a query string (in the postData variable), you can transmit that data through an asynchronous POST using the $.post()

method as follows:

$.post("vote.php", **postData**);

Step 5: Complete Control over AJA X

```
$.ajax(
    { url: "vote.php",
      data: $("#voteForm").serialize(),
      async: true,
      type: post
    }
  );
```

Cross-Origin Resource Sharing (CORS)

Cross-origin resource sharing (CORS) uses new headers in the HTML5 standard implemented in most new browsers. If a site wants to allow any domain to access its content through JavaScript, it would add the following header to all of its responses.

Access-Control-Allow-Origin: *

A better usage is to specify specific domains that are allowed, rather than cast the gates open to each and every domain. In our example the more precise header

Access-Control-Allow-Origin: www.funwebdev.com

5.4 Asynchronous File Transmission

Asynchronous file transmission is one of the most powerful tools for modern web applications.
Consider a simple form as defined below:

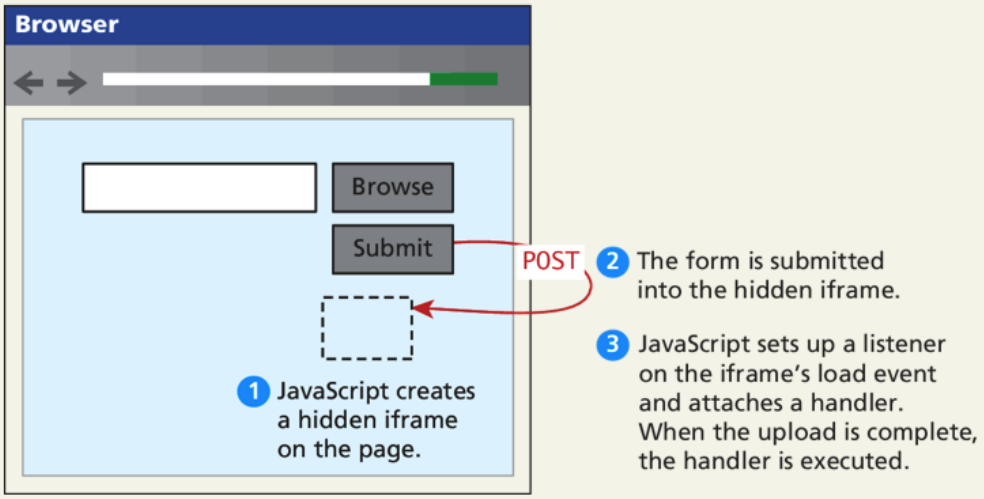
```
<form name="fileUpload" id="fileUpload" enctype="multipart/form-data"
      method="post" action="upload.php">
<input name="images" id="images" type="file" multiple />
<input type="submit" name="submit" value="Upload files!"/>
</form>
```

LISTING 15.17 Simple file upload form

Asynchronous File Transmission

The old iFrame technique

The original workaround to allow the asynchronous posting of files was to use a hidden <iframe> element to receive the posted files.

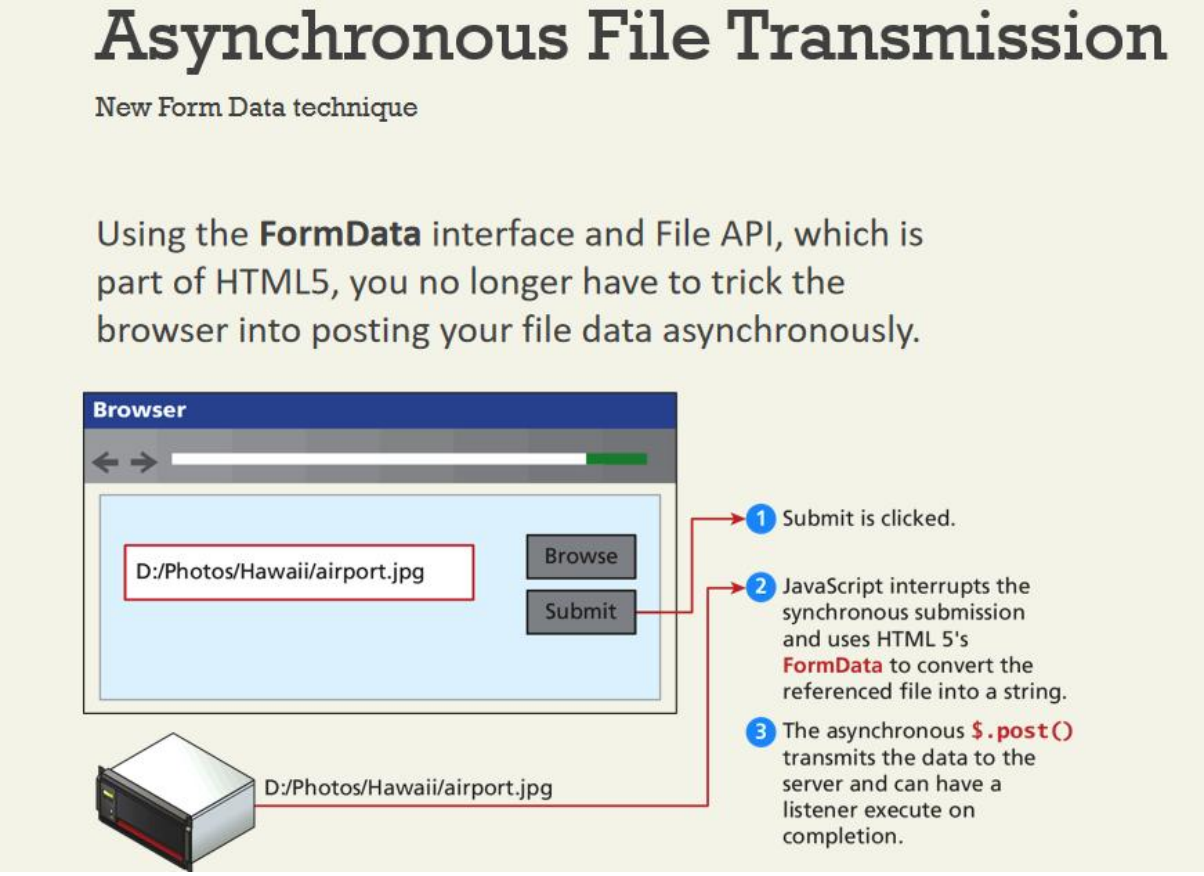



```
$(document).ready(function() {
  // set up listener when the file changes
  $(":file").on("change",uploadFile);
  // hide the submit buttons
  $("input[type=submit]").css("display","none");
});

// function called when the file being chosen changes
function uploadFile () {
  // create a hidden iframe
  var hidName = "hiddenIFrame";
  $("#fileUpload").append("<iframe id='"+hidName+"' name='"+hidName+"'
  style='display:none' src='#' ></iframe>");

  // set form's target to iframe
  $("#fileUpload").prop("target",hidName);
  // submit the form, now that an image is in it.
  $("#fileUpload").submit();
```

```
    // Now register the load event of the iframe to give feedback
    $('#'+hidName).load(function() {
    var link = $(this).contents().find('body')[0].innerHTML;
    // add an image dynamically to the page from the file just uploaded
    $("#fileUpload").append("<img src='"+link+"' />");
    });
}
```



5.5 XML Overview

XML is a markup language, but unlike HTML, XML can be used to mark up any type of data. Benefits of XML data is that as plain text, it can be read and transferred between applications and different operating systems as well as being human-readable.

XML is used on the web server to communicate asynchronously with the browser Used as a data interchange format for moving information between systems

What is XML?

XML stands for EXtensible Markup Language

XML is a markup language much like HTML

XML was designed to carry data, not to display data

XML tags are not predefined. You must define your own tags
XML is designed to be self-descriptive
XML is a W3C Recommendation

The Difference Between XML and HTML
XML is not a replacement for HTML.
XML and HTML were designed with different goals:
XML was designed to transport and store data, with focus on what data is.
HTML was designed to display data, with focus on how data looks.
HTML is about displaying information, while XML is about carrying information.

XML syntax

XML documents have syntactic and semantic structures. The syntax (think spelling and punctuation) is
made up of a minimum of rules such as but not limited to:
1. XML documents always have one and only one root element
2. Element names are case-sensitive
3. Elements are always closed
4. Elements must be correctly nested
5. Elements' attributes must always be quoted
6. There are only five entities defined by default (<, >, &, ", and ')
7. When necessary, namespaces must be employed to eliminate vocabulary clashes.

STRUCTURE OF XML LANGUAGE

```
//XML Declaration:
<?xml version="1.0" encoding="UTF-8"?>
//DTD Syntax
<!DOCTYPE root-element [<!element-declarations>]>
<!DOCTYPE website [
  <!ELEMENT website (name,company,phone)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT company (#PCDATA)>
  <!ELEMENT phone (#PCDATA)>
]>
// XML user document
<website>    //ROOT TAG
  <name>SUDARSANAN D</name>
```

```
  <company>CAMBRIEN SOFT PVT LTD</company>
  <phone>9535937675</phone>
</website>
```

There are only five entities defined by default

Certain characters in XML documents have special significance, specifically, the less than (<), greater than (>), and ampersand (&) characters. The first two characters are used to delimit the existence of element names. The ampersand is used to delimit the display of special characters commonly known as entities;

they ampersand character is the "escape" character. Consequently, if you want to display any of these three characters in your XML documents, then you must express them in their entity form:

- to display the & character type &
- to display the < character type <
- to display the > character type >

XML processors, computer programs that render XML documents, should be able interpret these characters without the characters being previously defined.

There are two other characters that can be represented as entity references:

- to display the ' character optionally type '
- to display the " character optionally type "

Defining XML vocabularies with DTDs

Creating your own XML mark up is all well and good, but if you want to share your documents with other people you will need to communicate to these other people the vocabulary your XML documents understand. This is the semantic part of XML documents -- what elements do your XML files contain and how are the elements related to each other? These semantic relationships are created using Document Type Definitions (DTD) and/or XML Schemas. DTDs are legacy implementations from the SGML world. They are more commonly used than the newer, XML-based, XML Schemas. This section provides an overview for creating DTDs.

DTDs can exist inside an XML document or outside an XML document. If they reside in an XML document, then they begin with a DOCTYPE declaration followed by the name of the XML document's root element and finally a list of all the elements and how they are related to each other. Here is a simple

DTD for embedded in the pets.xml file itself:

External DTD Declaration

If the DTD is declared in an external file, it should be wrapped in a DOCTYPE definition with

the following syntax:

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
 <to>Tove</to>
 <from>Jani</from>
 <heading>Reminder</heading>
 <body>Don't forget me this weekend!</body>
</note>
```

And this is the file "note.dtd" which contains the DTD:

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

XML Schemas:

DTDs have several disadvantages:

1. One is that DTDs are written in a syntax unrelated to XML, so they cannot be analyzed with an XML processor.
2. It can be confusing for people to deal with two different syntactic forms, one to define a document and one to define its structure.
3. DTDs do not allow *restrictions on the form of data* that can be the content of a particular tag.
4. for example, if the content of an element represents time, regardless of the form of the time data, a DTD can only specify that it is text, which could be anything. If fact, the content of an element could be an integer number, a floating point number, or a range of numbers. All of these would be specified as text. With DTDs, there are only ten data types, none of which is numeric.

Solution to solve this overcome problem are:

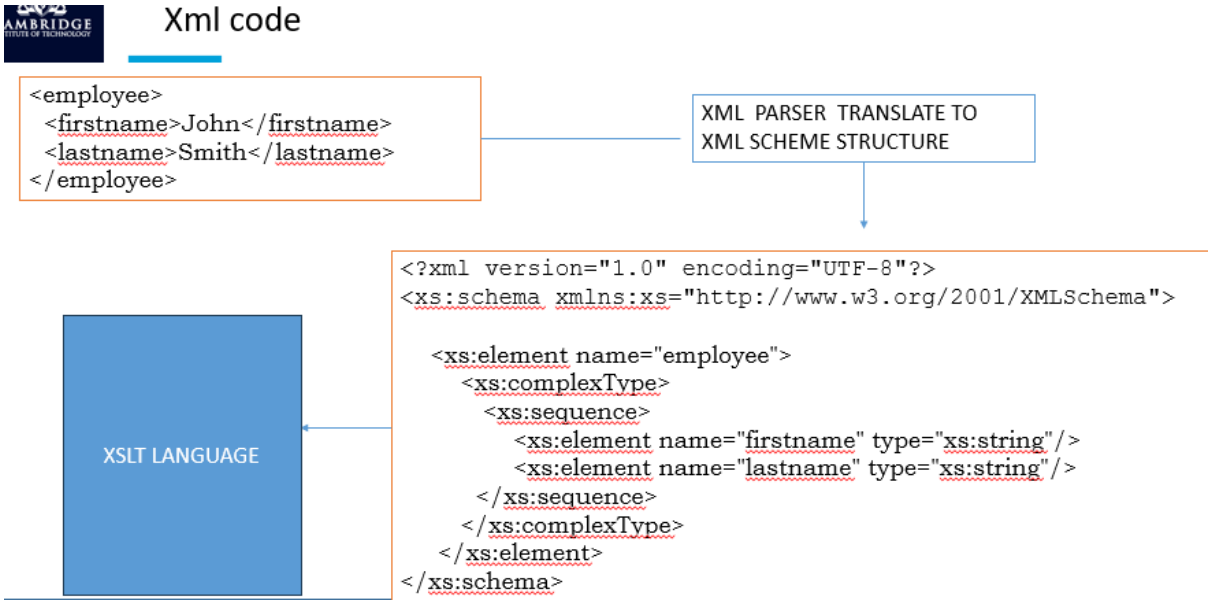
Several alternatives to DTDs have been developed to attempt to overcome their weaknesses**. XML schema**, which was designed by the W3C, is one of these alternatives.

An XML schema is an XML document, so it can be parsed with an XML parser. It also provides far more control over data types than do DTDs. The content of a specific element can be required to be any of 44 different data types.

An XML Schema:

- defines elements that can appear in a document
- defines attributes that can appear in a document
- defines which elements are child elements
- defines the order of child elements
- defines the number of child elements
- defines whether an element is empty or can include text
- defines data types for elements and attributes

defines default and fixed values for elements and attributes



<?xml version="1.0" encoding="UTF-8"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<!-- Root element definition -->

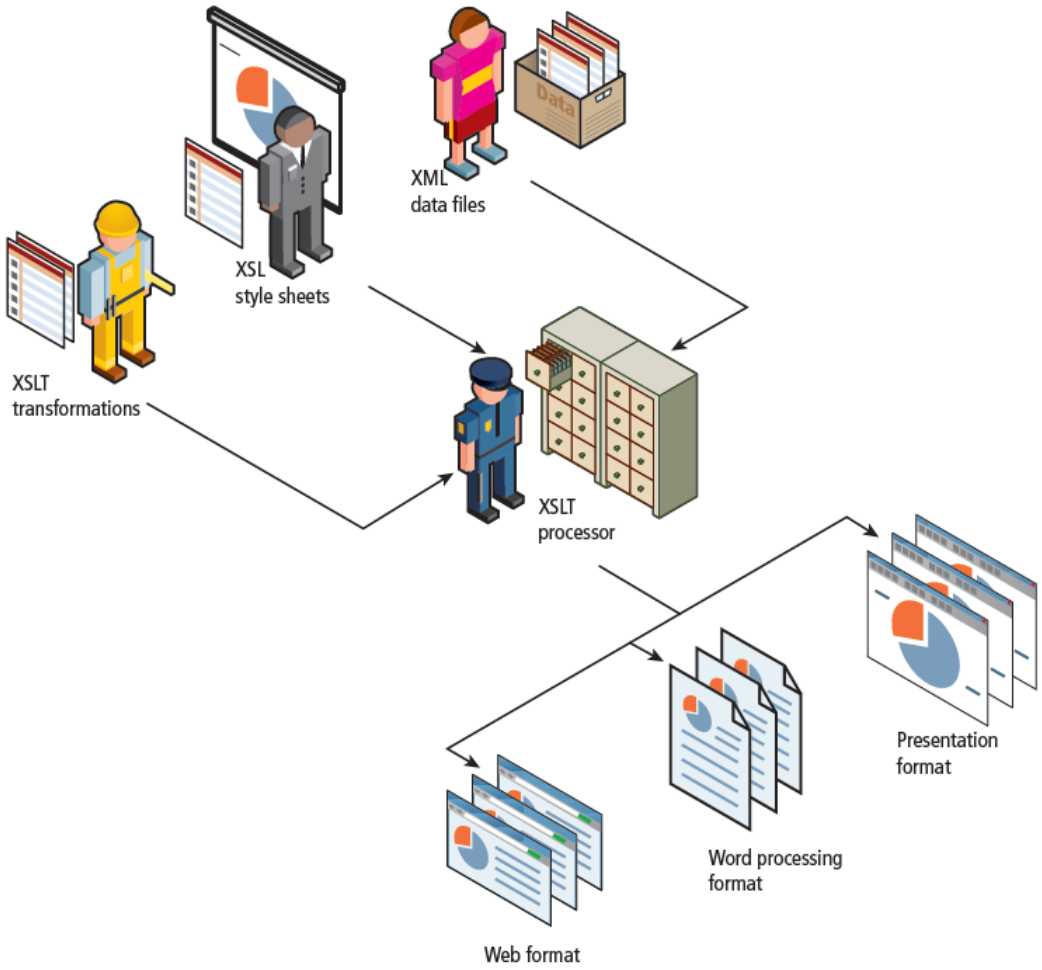
```
<xs:element name="bookstore">
  <xs:complexType>
   <xs:sequence>
    <!-- Nested element with a complex type structure -->
     <xs:element    name="book" maxOccurs="unbounded" minOccurs="1">
       <xs:complexType>
          <xs:sequence>
           <!-- Simple elements with built-in data types -->
           <xs:element name="title" type="xs:string"/>
           <xs:element name="author" type="xs:string"/>
           <xs:element name="year" type="xs:gYear"/>
           <xs:element name="price" type="xs:decimal"/>
           <!-- Attribute assigned to the book element -->
          <xs:attribute name="category" type="xs:string" use="required"/>
       </xs:complexType>
    </xs:element>
  </xs:sequence>
 </xs:complexType>
</xs:element>
</xs:schema>
```

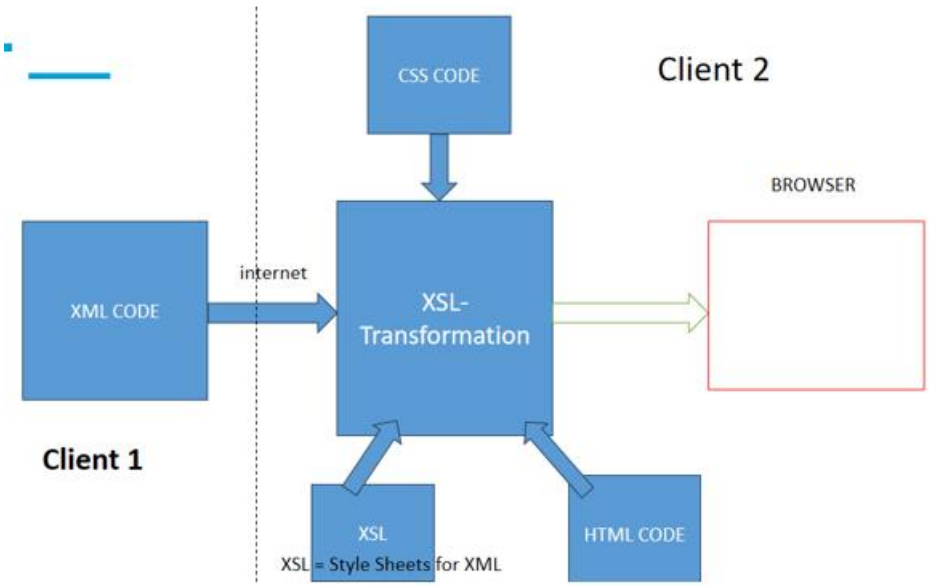
5.6 XSLT(XML Stylesheet Transformations)

The eXtensible sytlesheet language (XSL) is a family of recommendations for defining XML document transformations and presentation. It consists of three related standards:

1. XSL Transformations(XSLT)
2. XML path Language(Xpath)
3. XSL Formating Objects(XSL-FO)

Perhaps the most common translation is the conversion of XML to HTML. All of the modern browsers support XSLT, though XSLT is also used on the server side and within JavaScript





XML code[Art.xml]

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl"   href="Art.xsl"?>
<art>
   <painting id="290">
        <title>Balcony</title>
         <artist>
              <name>Sudarsanan D</name>
            <nationality>India</nationality>
         </artist>
         <year>1868</year>
        <medium>Oil on canvas</medium>
   </painting>
<art>
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<html xsl:version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns="http://www.w3.org/1999/xhtml">
<body>
    <h1>Catalog</h1>
    <ul>
      <xsl:for-each select="/art/painting">
      <li>
          <h2><xsl:value-of select="title"/></h2>
           <p>By: <xsl:value-of select="artist/name"/><br/>
           Year: <xsl:value-of select="year"/>
            [<xsl:value-of select="medium"/>]</p>
      </li>
    </xsl:for-each>
  </ul>
</body>
</html>
```

Art.xsl

XML path Language(Xpath):

Xpath is a language for expressions, which are often used to identify parts of XML documents, such as specific elements that are in specific positions in the document or elements that have particular attribute values. Xpath is also used for XML document querying languages, such as XQL , and for building new XML document structure using XPointer.

5.6 XML Processing

Two types

XML processing in PHP, JavaScript, and other modern development environments is divided into two basic styles:

- The **in-memory approach**, which involves reading the entire XML file into memory into some type of data structure with functions for accessing and manipulating the data.
- The **event or pull approach**, which lets you pull in just a few elements or lines at a time, thereby avoiding the memory load of large XML files.

XML Processing using javascript

All modern browsers have a built-in XML parser and their JavaScript implementations support an in-memory XML DOM API.

You can use the already familiar DOM functions such as

- getElementById(),
- getElementsByTagName()
- createElement()

to access and manipulate the data.

```
<script>
if (window.XMLHttpRequest)  {
  // code for IE7+, Firefox, Chrome, Opera, Safari
  xmlhttp=new XMLHttpRequest();
}
else  {
  // code for old versions of IE (optional you might just decide to
  // ignore these)
  xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
}
```

```
// load the external XML file
xmlhttp.open("GET","art.xml",false);
xmlhttp.send();
xmlDoc=xmlhttp.responseXML;

// now extract a node list of all <painting> elements
paintings = xmlDoc.getElementsByTagName("painting");
if (paintings) {
   // loop through each painting element
   for (var i = 0; i < paintings.length; i++)
   {
      // display its id attribute
      alert("id="+paintings[i].getAttribute("id"));

      // find its <title> element
      title = paintings[i].getElementsByTagName("title");
      if (title) {
         // display the text content of the <title> element
         alert("title="+title[0].textContent);
      }
   }
}
</script>
```

LISTING 17.5 Loading and processing an XML document via JavaScript

XML Processing using jQuery

```
art = '<?xml version="1.0" encoding="ISO-8859-1"?>';
art += '<art><painting id="290"><title>Balcony … </art>';

// use jQuery parseXML() function to create the DOM object
xmlDoc = $.parseXML( art );
// convert DOM object to jQuery object
$xml = $( xmlDoc );

// find all the painting elements
$paintings = $xml.find( "painting" );
// loop through each painting element
$paintings.each(function() {
   // display its id
   alert($(this).attr("id"));
   // find the title element within the current painting element
   $title = $(this).find( "title" );
   // and display its content
   alert( $title.text() );
});
```

LISTING 17.6 XML processing using jQuery

XML Processing using php

PHP provides several extensions or APIs for working with XML including:

- The **SimpleXML** extension which loads the data into an object that allows the developer to access the data via array properties and modifying the data via methods.
- The **XMLReader** is a read-only pull-type extension that uses a cursor-like approach similar to that used with database processing

```php
<?php

$filename = 'art.xml';
if (file_exists($filename)) {
   $art = simplexml_load_file($filename);

   // access a single element
   $painting = $art->painting[0];
   echo '<h2>' . $painting->title . '</h2>';
   echo '<p>By ' . $painting->artist->name . '</p>';
   // display id attribute
   echo '<p>id=' . $painting["id"] . '</p>';

   // loop through all the paintings
   echo "<ul>";
   foreach ($art->painting as $p)
   {
      echo '<li>' . $p->title . '</li>';
   }
   echo '</ul>';
} else {
   exit('Failed to open ' . $filename);
}

?>
```

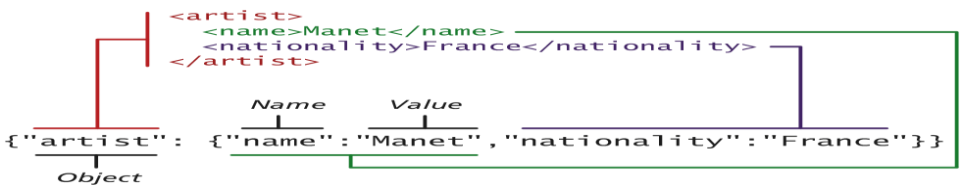
LISTING 17.8 Using simple XML

5.7 JSON (javasript Object Notatin)

JSON stands for JavaScript Object Notation (though its use is not limited to JavaScript)

Like XML, JSON is a data serialization format. It provides a more concise format than XML.

Many REST web services encode their returned data in the JSON data format instead of XML.



Using JSON in JavaScript

Creating JSON JavaScript objects

it is easy to make use of the JSON format in JavaScript:

```
var a = {"artist": {"name":"Manet","nationality":"France"}};

alert(a.artist.name + " " + a.artist.nationality);
```

When the JSON information will be contained within a string (say when downloading) the JSON.parse() function can be used to transform the string containing into a JavaScript object

Using JSON in JavaScript

Convert string to JSON object and vice versa

```
var text = '{"artist": {"name":"Manet","nationality":"France"}}';

var a = JSON.parse(text);

alert(a.artist.nationality);
```

JavaScript also provides a mechanism to translate a JavaScript object into a JSON string:

```
var text = JSON.stringify(artist);
```

Converting a JSON string into a PHP object is quite straightforward:

```php
<?php
  // convert JSON string into PHP object
  $text = '{"artist": {"name":"Manet","nationality":"France"}}';
  $anObject = json_decode($text);
  echo $anObject->artist->nationality;

  // convert JSON string into PHP associative array
  $anArray = json_decode($text, true);
  echo $anArray['artist']['nationality'];
?>
```

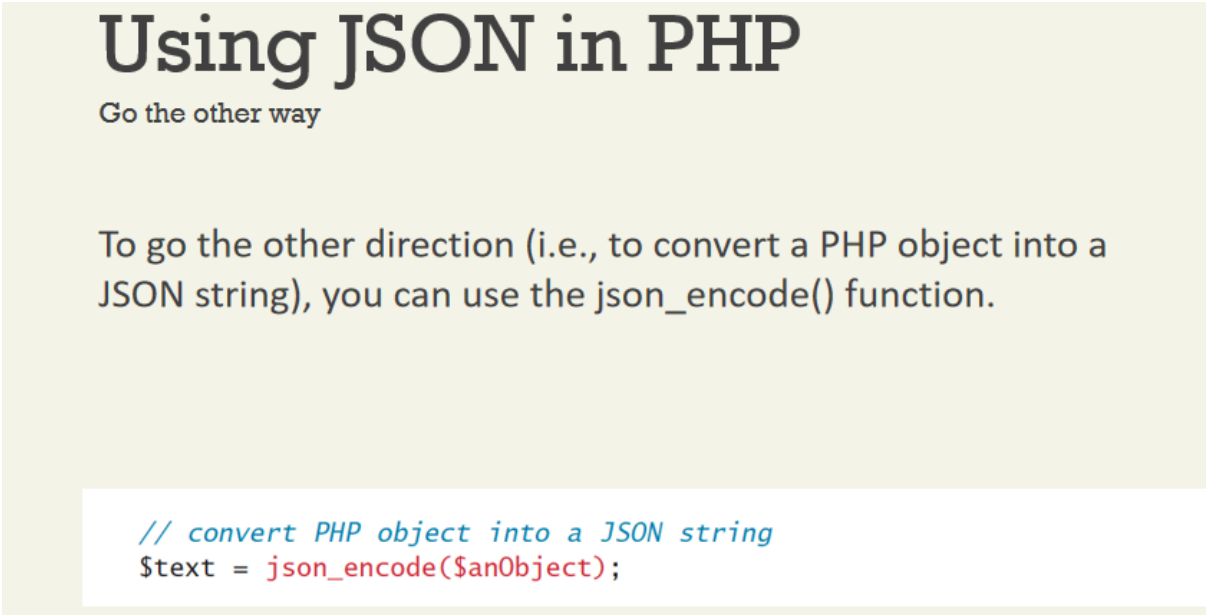
Notice that the json_decode() function can return either a PHP object or an associative array.

Converting a JSON string into a PHP object is quite straightforward:

```php
<?php
  // convert JSON string into PHP object
  $text = '{"artist": {"name":"Manet","nationality":"France"}}';
  $anObject = json_decode($text);
  echo $anObject->artist->nationality;

  // convert JSON string into PHP associative array
  $anArray = json_decode($text, true);
  echo $anArray['artist']['nationality'];
?>
```

Notice that the json_decode() function can return either a PHP object or an associative array.



5.8 Overview of Web Services

Web services are the most common example of a computing paradigm commonly referred to as **service-oriented computing** (SOC).

A **service** is a piece of software with a platform-independent interface that can be dynamically located and invoked.

Web services are a relatively standardized mechanism by which one software application can connect to and communicate with another software application using web protocols.

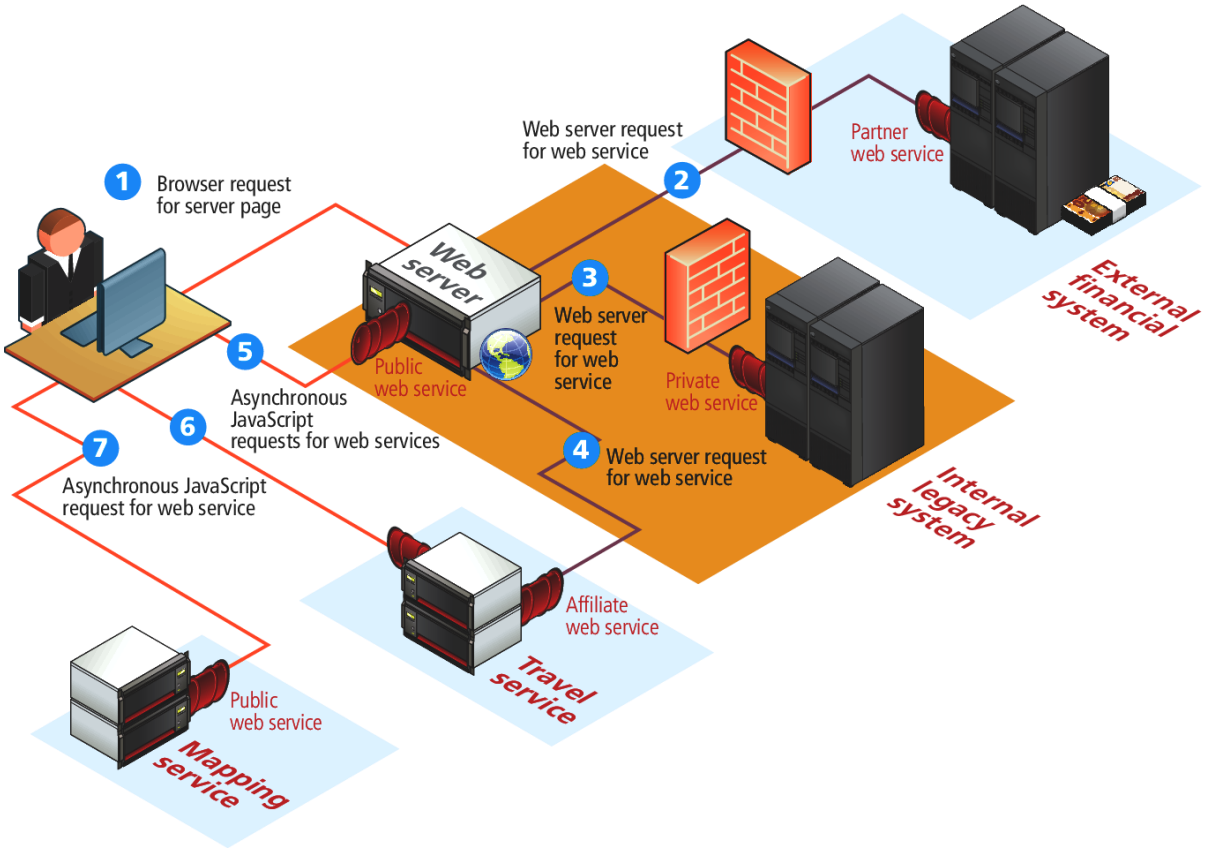
- A web service is a standardized, machine-to-machine software system that allows different applications to communicate and exchange data over a network.
- By using open, universal standards like HTTP, XML, and JSON, web services ensure that programs written in entirely different languages or on different platforms can seamlessly interoperate

Major Types of Web Services (benefit)

RESTful Web Services (REST-Representational State Transfer **):** The modern industry standard. REST uses standard HTTP methods (GET, POST, PUT, DELETE) and is highly lightweight. It typically exchanges data via JSON, making it fast and easy to implement.

1. **SOAP (service-oriented architecture protocol OR** Simple Object Access Protocol**) Web Services:** A highly structured, protocol-driven approach that relies strictly on XML for message formatting. It includes built-in error handling and strict security (WS-Security), making it common in enterprise and financial environments.

2. **GraphQL:** A newer query language for APIs that allows clients to request exactly the specific data they need, reducing inefficient data transfers.
3. **gRPC:** A high-performance, open-source framework developed by Google, frequently used for connecting microservices quickly.



Why do we need a web service?

Web-based apps are developed using a range of programming platforms in today's corporate world. Some applications are written in Java, others in .Net, and still others in Angular JS, Node.js, and other frameworks. Most of the time, these diverse programs require some form of communication to work together. Because they are written in separate programming languages, ensuring accurate communication between them becomes extremely difficult. Web services have a role in this. Web services provide a common platform for several applications written in different programming languages to connect with one another

For web services, what kind of security is required?

Web services should have a higher level of security than the Secure Socket Layer (SSL) (SSL). Entrust Secure Transaction Platform is the only way to attain this level of security. This level of security is required for web services in order to assure dependable transactions and secure confidential information.

SOAP vs REST

TYPE	SOAP (Simple Object Access Protocol)	REST (Representational State Transfer)
Type	Protocol with strict standards	Architectural style with flexible guidelines
Data format	XML only	Multiple formats: JSON, XML, HTML, plain text
Complexity	More complex and rigid	Simpler and lightweight
Performance	Slower due to XML and overhead	Faster, especially with JSON
Security	Built-in security (WS-Security, encryption, authentication)	Relies on HTTPS and external security measures
Statefulness	Can be stateful or stateless	Stateless by design
Transport protocols	Works over HTTP, SMTP, TCP, etc.	Primarily uses HTTP/HTTPS
Error handling	Standardized error handling	Uses HTTP status codes
Best use cases	Enterprise systems, banking, high-security transactions	Web apps, mobile apps, public APIs
Ease of implementation	Requires more setup and strict rules	Easier to implement and integrate

What are the real-world examples and applications of web services?

Web services power much of our daily digital experience, often without us realizing it.

- **E-commerce:** Online stores query inventory web services to check product availability and real-time stock levels before confirming purchases.
- **Travel industry:** Booking sites aggregate flight, hotel, and car rental data from multiple provider web services to show real-time options and prices.
- **Finance:** Apps use web services for live currency exchange rates, stock prices, and market data from financial exchanges.
- **Weather apps:** Mobile apps request location-based weather services to display current conditions and forecasts.